

Project A

Submission Instructions

1. Submission Date: 12.06.26, at 22:55.
2. Please contact Omer (omer.y@campus.technion.ac.il) for any administrative or professional question. Questions asked via the forum will always be answered faster than those sent via email. Professional questions shall be asked in English.
3. Team Submission: only pairs, unless otherwise approved in advance.
4. Submission File Format: Each team should submit a zip file named `<ID1_ID2>`, where `ID<i>` represents the ID number of each of the two team members. The zip file must include the following files:
 - For Section A, submit **only** the file `vector_index.py`. Do not submit any additional generated files, evaluation outputs, public scenarios, or modified support files.
 - For Section B, submit a link to a **public** GitHub repository (see Section 2). Put the link in a `txt` file called *Repo Link*.
5. Code Compliance: Ensure that your code adheres to the provided template.
6. It is highly recommended to use the server assigned for you. Please follow the instructions provided in the files `Students - How to work with your VM.pdf` and `installations.txt`.
7. If you struggle with package installations, please notify Omer as soon as possible via the forum.
8. The grade of each section constitutes 50% of the overall grade.
9. Cheating, plagiarism and copying code or reports from other students are strictly forbidden and will be handled according to the Technion academic regulations.

1 Section A

In this section, you are required to implement a dynamic vector database index that supports insertion, deletion and similarity search operations over continuously changing data. The goal is to design a data structure that can efficiently maintain and query high-dimensional vectors under dynamic updates.

You are provided with a student bundle containing source files, public evaluation scenarios and scripts for self-evaluation. The evaluation process simulates realistic vector database behavior in which data arrives in batches, data distributions gradually shift over time, and vectors may later be deleted.

Provided Files The provided student package contains the following files:

- `vector_index.py`: the file you must submit. Implement the `VectorIndex` class here.
- `student_vector_index_skeleton.py`: reference API and comments (do not submit).
- `naive_vector_index.py`: reference **naive** implementation (dict storage + full scan). Used only to measure runtime on **your** machine. **Do not modify**.
- `evaluation.py`: evaluation utilities (read-only).
- `scripts/`: scripts for evaluating your implementation on public scenarios.
- `data/public/`: public test scenarios.

You should not modify any file except `vector_index.py`. This is the only file you submit.

Import restriction: You may only use packages already imported in this file. Importing additional external packages inside `vector_index.py` is not allowed and may result in disqualification of this section because the automatic tests are likely to fail.

The complete student package for this section can be downloaded from the following reference:

Section A Download Link

Required Implementation You are required to implement the `VectorIndex` class, containing the three functions: `insert`, `delete` and `search`, described as follows:

Insert Operation The `insert` function receives a mapping `Dict[int, np.ndarray]` from `vector_id` to `vector`. Insertion of a vector succeeds only if no vector with the same ID is already in the index. The function returns:

```
{
"succeeded": [...],
"failed": [...]
}
```

listing IDs inserted successfully and IDs that failed, preserving the order of IDs within each list as produced by your implementation for that batch.

Delete Operation The delete function receives `np.ndarray` of vector IDs. Deletion succeeds only if the vector currently exists. Deleting a non-existing ID shall not raise an exception (it is recorded as failed). The function returns the same `succeeded / failed` structure as insert.

Search Operation The search function receives:

- query vectors of shape `(num_queries, dim)`;
- an integer `k`.

It returns an array of shape `(num_queries, min(k, n_active))`, where entry (i, j) is the ID of the j -th nearest neighbor to query i (0-index). Returned IDs must be sorted by **descending** dot-product similarity (highest first). Vectors are already L2-normalized; similarity is the dot product.

Scenario Structure Three **public** scenarios are provided for debugging and self-evaluation. Each scenario is a directory `data/public/scenario_XX/` with a fixed sequence of operations. Search queries partially overlap across stages: neighbors may be inserted or deleted later, so expected nearest neighbors can change over time. file types are:

- `batch*.npy`: insertion batches (object arrays of `(id, vector)` pairs)
- `delete*.npy`: IDs to delete
- `query*.npy`: query vectors
- `query*_gt.npy`: ground-truth neighbor IDs (top-10 per query; evaluation uses top-10)
- `step*_insert_gt.json, step*_delete_gt.json`: expected insert/delete outcomes
- `manifest.json`: ordered list of operations (insert / delete / search)
- `.baseline_local.json`: created on your machine after the first local baseline run (see below); safe to delete to force re-calibration.

The evaluator replays operations in the order given by `manifest.json`.

Note: `manifest.json` may contain instructor-machine timing fields (`baseline_initial_time`, `baseline_dynamic_time`, `baseline_total_time`). **Self-evaluation does not use them.**

Runtime is compared to a dynamic-phase baseline measured on **your** computer (see Scoring).

If you have an old `.baseline_local.json` without `cache_version`, pass `--recalibrate-baseline` once.

Running Self-Evaluation From the `student/` directory:

```
cd student
python scripts/eval_scenario_1.py
```

Similarly `eval_scenario_2.py` and `eval_scenario_3.py`, or:

```
python scripts/eval_scenario.py --scenario 3
```

What each run does Each script performs two steps:

1. **Local naive baseline.** The reference implementation in `naive_vector_index.py` replays the full scenario **three times**; the average **dynamic-phase** time (all operations after the initial bulk insert) is the baseline used for runtime scoring on your machine.
2. **Your implementation.** `vector_index.py` replays the same scenario once; correctness and phase times are reported.

On the **first** run for a scenario, step (1) is slow (about three full scenario replays). The result is saved in:

```
data/public/scenario_XX/.baseline_local.json
```

Later runs read this cache and skip re-measuring unless you pass:

```
python scripts/eval_scenario_1.py --recalibrate-baseline
```

Use `--recalibrate-baseline` after changing hardware, OS power settings, or if you suspect a stale cache.

The message `Measuring local naive baseline (3 runs)...` is printed every time; if a cache exists, the value may appear immediately without re-running the three naive replays.

Scoring Policy (Public Self-Evaluation) Scores are computed by `evaluation.py` as follows.

Correctness

- **Insert / delete score** (each in $[0, 1]$): For every batch, compare your `succeeded` and `failed` ID lists to the ground truth using Jaccard similarity; average over batches weighted by batch size.
- **Search score:** For each hidden scenario, the search quality is measured using the average Recall@10 over all search operations in the scenario.

Let $\text{Recall}_{\text{team}}$ denote the search score of the current submission, and let $\text{Recall}_{3\text{rd}}$ denote the search score of the third-best submission on the hidden set.

The final search component is assigned relatively:

- 1st place: 100% of the search weight + 5 bonus points (for the search component).
- 2nd place: 100% + 3 bonus points.
- 3rd place: 100%.
- 4th place and below:

$$\min\left(1, \frac{\text{Recall}_{\text{team}}}{\text{Recall}_{3\text{rd}}}\right) \times 100\%.$$

- **Functional score:**

$$\text{functional} = 0.5 \cdot \text{search} + 0.3 \cdot \text{insert} + 0.2 \cdot \text{delete}.$$

Runtime Only time spent inside `insert`, `delete`, and `search` is measured. The **initial bulk insert** (first insert in the scenario, batch 0) is reported but **not** included in the runtime penalty (offline-style build, analogous to Section B index build). The **dynamic phase** comprises later inserts, all deletes, and all searches.

Let T_{dyn} be your dynamic-phase time and T_{base} the matching local naive average from step (1). Define the **speed ratio**

$$\rho = T_{\text{dyn}}/T_{\text{base}}.$$

Wall-clock cap (entire scenario). The full replay – initial bulk insert, later inserts, deletes, and searches – must complete within **50 seconds** wall clock on every machine (same limit in hidden grading). If the scenario exceeds 50s, or any operation is skipped because the cap was already exceeded, the **runtime multiplier is set to 0.5** (minimum). Baseline calibration replays are exempt from this cap.

The runtime multiplier is defined as follows:

$$M(\rho) = \begin{cases} 1, & \rho \leq 0.2 \\ 1 - \frac{0.3}{0.3}(\rho - 0.2), & 0.2 < \rho \leq 0.5 \\ 0.7 - \frac{0.2}{0.5}(\rho - 0.5), & 0.5 < \rho \leq 1 \\ 0.5, & \rho > 1 \end{cases}$$

Thus:

- Solutions that are at least $5\times$ faster than the naive baseline receive no runtime penalty.
- Between $\rho = 0.2$ and $\rho = 0.5$, the multiplier decreases linearly from 1 to 0.7.
- Between $\rho = 0.5$ and $\rho = 1$, it further decreases linearly from 0.7 to 0.5.
- Solutions slower than the naive baseline receive a multiplier of 0.5.

Final scenario score

$$\text{final_score} = \text{functional_score} \times \text{runtime_multiplier}.$$

Final Grading The course autograder will use **three additional hidden scenarios** with the same API and scoring logic. Design a solution that generalizes; do not overfit the public scenarios. For each hidden scenario, a scenario score is computed exactly as described above, including correctness and runtime components. The final score for Section A is:

$$\text{SectionAScore} = \frac{\text{ScenarioScore}_1 + \text{ScenarioScore}_2 + \text{ScenarioScore}_3}{3}$$

namely, the average score across the three hidden scenarios. Hidden evaluation will also use a **machine-local naive baseline** (same idea as self-evaluation), not a fixed time embedded in `manifest.json`. Finally, after computing the average scenario score, the line-count penalty multiplier described earlier is applied.

2 Section B

In this section, you implement an **end-to-end retrieval pipeline** over a collection of textual Wikipedia-style entries. Your system shall receive a **batch of queries** and return, for each query, a ranked list of relevant `page_id` values.

Unlike Section A (dynamic vector *index* operations), you own the full pipeline: preprocessing, optional chunking, embedding, offline indexing, and query-time retrieval. The corpus is partially synthetic; facts may be fictional. Queries are built over this corpus so the complete relevant set per query is known. A query may have **multiple** relevant pages.

Student package – files you receive Download the Section B bundle from link below. Unpack the `student/` folder. You should have:

- `main.py` – implements `run(queries)` called by the autograder.
- `chunk.py`, `embed.py`, `index.py`, `retrieve.py`, `utils.py` – pipeline modules you may edit.
- `eval.py` – **read-only** NDCG@10 utilities.
- `requirements.txt`, `README.md`.
- `scripts/eval_public.py` – self-test on 50 public queries (**read-only**).
- `scripts/build_index.py` – local offline index build (**read-only**; for your machine only).
- `data/public_queries.json` – 50 public queries with labels.
- `data/Wikipedia Entries/` – one JSON file per page (full retrieval corpus).
- `artifacts/` – directory for your precomputed index and embeddings (empty in the handout; **required in your submission**).

Not included in the student zip:

- `hidden_queries.json` – instructor/autograder only (you never receive it).

You may modify every file in `student/` **except** `eval.py`, `scripts/eval_public.py`, and `scripts/build_index.py`.

Section B Download Link

Corpus and JSON format Each page file:

```
{
  "page_id": 25051,
  "title": "...",
  "content": "..."
}
```

`page_id` is an integer. Public queries:

```
{
  "query_id": "q_public_001",
  "query": "...",
  "relevant_page_ids": [20263, 9112]
}
```

Relevance is binary. Retrieval is over **all** files in `data/Wikipedia Entries/`.

Required API The autograder calls **once** with all evaluation queries:

```
def run(queries: list[str]) -> list[list[int]]:
```

Return one ranked list of **page_id** (**int**) per query, most relevant first. Only the first 10 IDs per list are scored. Duplicate IDs after the first occurrence and invalid IDs earn no extra gain.

Embeddings and imports Use `sentence-transformers/all-MiniLM-L6-v2` for all retrieval embeddings.

Import restriction: only the standard library and `requirements.txt`: `numpy`, `sentence-transformers`, `faiss-cpu` (or `faiss`).

Submitted index (required) You must build the corpus index **offline on your own machine** and submit the resulting files under `artifacts/` in your public GitHub repository. The autograder **does not** run `scripts/build_index.py` or call `build_offline_index()` at grading time; it only calls `run(queries)` and expects your code to load the prebuilt index from disk.

Include every file your `run()` implementation needs (e.g. vector matrices, FAISS indexes, metadata JSON). Document each artifact path and format in your README. Custom **trained** checkpoints must be in the repo; pretrained MiniLM weights need not be shipped if loaded from the hub. Use Git LFS if files are large.

Local setup: document `pip install -r requirements.txt` and index build in your README for your own machine. **Grading** assumes dependencies are installed and only runs your code (see GitHub rubric below).

Pipeline and timing Design freely (chunking, FAISS, reranking, etc.). Grading is at `page_id` level.

- **Offline (untimed, your machine):** corpus embedding and index build (`python scripts/build_index.py`). Not run by staff at grading time.
- **Graded (≤ 60 s, GPU):** one call `run(all_queries)` including query embedding and retrieval over your submitted `artifacts/`.

Self-evaluation

```
pip install -r requirements.txt
python scripts/build_index.py
python scripts/eval_public.py
```

Prints mean NDCG@10 on public queries only. Hidden queries are not distributed.

Section B grade Your Section B score has three weighted components; creativity is judged separately as a possible bonus on the **total** Section B score (not a fixed weight):

$$\text{Section B} = 0.5 \times \text{Functional} + 0.25 \times \text{GitHub} + 0.25 \times \text{Video}.$$

Submit a **public GitHub** repository and put a clear link to your presentation video in the README.

Functional (50% of Section B) Mean NDCG@10 over 50 hidden queries (binary relevance). **Relative ranking** on the hidden set applies *inside* this component; you see only your own result after grading:

- 1st: 100% of this component + 5 bonus points on Section B.
- 2nd: 100% + 3 bonus points on Section B.
- 3rd: 100%.
- 4th+: $\min(1, \text{NDCG}_{\text{team}} / \text{NDCG}_{\text{3rd}}) \times 100\%$ of this component.

Missing or incompatible **artifacts/** prevent running `run()` and yield 0 for this part.

GitHub repository (25% of Section B) Staff grade the repo on a 0–100 rubric, then scale to 25% of Section B:

- **50%** — seamless run: with dependencies already installed (no `pip install` at grading), `python scripts/eval_public.py` succeeds on a fresh clone **without** rebuilding the index.
- **20%** — modular code: small functions, logical split across files.
- **10%** — clear README (setup, artifacts, how to run eval).
- **10%** — clear in-code documentation.
- **10%** — evident pair collaboration: both members have meaningful commits in the history (not a single-author dump).

Video presentation (25% of Section B) **Requirements:** max **3:00**; at most **10 slides**; **both** team members speak; link in README. For **each** pipeline stage (**chunk**, **embed**, **index**, **retrieve**), explain your method. Show the **process** you followed and **empirical results** (plots/metrics) that justify your final design. Do **not** present by scrolling through code, a manuscript, or pasted code on slides.

Staff grade on a 0–100 rubric, then scale to 25% of Section B. **Penalties** (deducted from the video subscore only): 0.1 point per second over 3:00; 0.1 point per slide over 10.

- **30%** — clarity: we understand your method and pipeline choices.
- **20%** — empirical discussion: decisions backed by results you obtained during development (shown visually).
- **20%** — slides: concise and clear (within the 10-slide limit).
- **20%** — both members participate meaningfully.
- **10%** — time management (within 3:00, before penalties).

Creativity bonus Not part of the formula above. The **most creative** submission (staff judgment, informed mainly by the video) may receive an extra bonus on your **total** Section B score.